

Experiment: TCP Echo Client–Server using Socket Programming

Aim

To implement a TCP echo client and server using socket programming, where the server sends back the same message received from the client.

Algorithm

Algorithm for Echo Server

1. Start the program
 2. Create a TCP socket using `socket()`
 3. Bind the socket to a specific IP address and port number using `bind()`
 4. Listen for incoming client connections using `listen()`
 5. Accept the client connection using `accept()`
 6. Receive data from the client using `recv()`
 7. Send the same data back to the client using `send()`
 8. Repeat steps 6 and 7 until the client disconnects
 9. Close the client socket and server socket
 10. Stop the program
-

Algorithm for Echo Client

1. Start the program
2. Create a TCP socket using `socket()`
3. Specify the server IP address and port number
4. Connect to the server using `connect()`
5. Read a message from the user
6. Send the message to the server using `send()`
7. Receive the echoed message from the server using `recv()`
8. Display the received message

9. Repeat steps 5 to 8 as required
10. Close the socket
11. Stop the program

Function Explanation

socket()

- Creates a communication endpoint (socket).
- Used by both client and server.
- Syntax:

```
int socket(int domain, int type, int protocol);  
example: socket(AF_INET, SOCK_STREAM, 0);
```

bind()

- Associates the socket with an IP address and port number.
- Used only by the server.
- Syntax:

```
int bind(int sockfd, struct sockaddr *addr, socklen_t addrlen);
```

listen()

- Puts the server socket in passive mode to wait for client connections.
- Syntax:

```
int listen(int sockfd, int backlog);
```

accept()

- Accepts an incoming client connection.
- Creates a new socket for communication with the client.
- Syntax:

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

connect()

- Used by the client to establish a connection with the server.
- Syntax:

```
int connect(int sockfd, struct sockaddr *addr, socklen_t addrlen);
```

send()

- Sends data from one socket to another.
- Syntax:

```
int send(int sockfd, const void *buf, size_t len, int flags);
```

recv()

- Receives data from a socket.
- Syntax:

```
int recv(int sockfd, void *buf, size_t len, int flags);
```

close()

- Closes the socket and releases resources.
- Syntax:

```
int close(int sockfd);
```

Conclusion: